Laboratory — Logical Reasoning for Planning

Using an LLM to Construct and Test a Simple Planning Agent

Undergraduate Artificial Intelligence • Shrihari Karthik • f20230868@goa.bits-pilani.ac.in

Contents of this submission

#	Required item	Where it is
1	Planning problem specification	§1 (Task 0)
2	Manually constructed plan + states	§2 (Task 1)
3	Exact LLM prompt used	§3 (Task 2)
4	Generated Python planner	`planner.py` (separate file)
5	Results of at least 3 tests	§4 (Task 3)
6	Answers to "Think About It" questions	§1, §3, §5 (boxed)
7	Reflection answers	§7
–	LLM provenance statement	§8

1. Task 0 — Understanding the Planning Problem

(a) Initial state I

$$I = \{\text{At}(\mathit{Robot}, A), \text{At}(\mathit{Package}, A)\}$$

Under the closed-world assumption, every atom not listed is false. In particular Holding($\mathit{Package}$) is false in I, and the robot is not at B or C.

(b) Goal G

$$G = \{\text{At}(\mathit{Package}, C)\}$$

The goal constrains only the package. It says nothing about where the robot ends up, and nothing about whether the robot is holding anything. Any state containing At($\mathit{Package}, C$) is a goal state.

(c) Available actions

The warehouse graph is undirected with edges A–B and B–C; there is no direct edge A–C. Ground actions:

- Move: `Move(A,B)`, `Move(B,A)`, `Move(B,C)`, `Move(C,B)` — four ground instances.
- PickUp: `PickUp(Package,A)`, `PickUp(Package,B)`, `PickUp(Package,C)`.

- Drop: `Drop(Package,A)`, `Drop(Package,B)`, `Drop(Package,C)`.

Ten ground actions in total.

(d) Preconditions and effects

Action	Positive preconditions	Negative preconditions	Positive effects (add)	Negative effects (delete)
`Move(x,y)` for `Connected(x,y)`	$\mathrm{At}(\mathit{Robot}, x)$	—	$\mathrm{At}(\mathit{Robot}, y)$	$\mathrm{At}(\mathit{Robot}, x)$
`PickUp(p,l)`	$\mathrm{At}(\mathit{Robot}, l)$, $\mathrm{At}(p, l)$	$\mathrm{Holding}(p)$ (added assumption, see below)	$\mathrm{Holding}(p)$	$\mathrm{At}(p, l)$
`Drop(p,l)`	$\mathrm{At}(\mathit{Robot}, l)$, $\mathrm{Holding}(p)$	—	$\mathrm{At}(p, l)$	$\mathrm{Holding}(p)$

Assumptions made.

(A1) A state is a set of ground atoms with the closed-world assumption.

(A2) The robot has one gripper, encoded as the negative precondition $\neg\,\mathrm{Holding}(p)$ on PickUp — the laboratory text does not state this, but the LLM specification asks for negative preconditions and this is the natural place for one.

(A3) Effects are applied delete-first, then add.

(A4) The goal is a set of positive atoms and $S \models G$ iff $G \subseteq S$.

(A5) All actions are ground; no unification is performed.

(A6) Every action has unit cost, so BFS returns a plan of minimum length.

Which actions are applicable in I?

Applying the test $S \models \mathrm{Preconditions}(a)$ to each of the ten ground actions gives exactly two applicable actions (this is confirmed by the program's own Task 0 output, reproduced in §4).

Action	Applicable?	Reason
`PickUp(Package,A)`	**YES**	Both preconditions $\text{At}(\mathit{Robot}, A)$ and $\text{At}(\mathit{Package}, A)$ are in I, and $\text{Holding}(\mathit{Package}) \notin I$, so the negative precondition is also satisfied.
`Move(A,B)`	**YES**	Its only precondition $\text{At}(\mathit{Robot}, A)$ is in I.
`Drop(Package,C)`	**NO**	Both preconditions fail: $\text{At}(\mathit{Robot}, C) \notin I$ (the robot is at A) and $\text{Holding}(\mathit{Package}) \notin I$ (the package is on the floor at A, not in the gripper).
`Move(B,A)`, `Move(B,C)`, `Move(C,B)`	**NO**	Each requires the robot to be at B or C; the robot is at A.
`PickUp(Package,B)`, `PickUp(Package,C)`	**NO**	Require $\text{At}(\mathit{Robot}, \cdot)$ and $\text{At}(\mathit{Package}, \cdot)$ at B or C; neither holds.
`Drop(Package,A)`, `Drop(Package,B)`	**NO**	Require $\text{Holding}(\mathit{Package})$, which is false in I.

Think About It (Task 0)
"An action should not be considered applicable merely because it appears in the list of available actions. Ask: are all of its preconditions satisfied in the current state?"
The action set A is a static description of what the robot is capable of; it is not a description of what the robot may do now. Membership in A is a syntactic fact about the domain, whereas applicability is a semantic fact about the current state: it is the entailment $S \models \text{Preconditions}(a)$.
`Drop(Package,C)` is in A at every instant of the plan, yet it is applicable only in the single state S_3. Confusing the two is exactly the failure mode that produces a plan which reads plausibly but cannot be executed — and it is the point at which logical reasoning, rather than pattern matching, enters planning.

2. Task 1 — A Plan Constructed by Hand

A plan is a sequence $a_1, \ldots, a_n$ with

$$I \xrightarrow{a_1} S_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} S_n$$

and

$$S_n \models G.$$

Reasoning backwards: to get $\text{At}(\mathit{Package}, C)$ the last action must be `Drop(Package,C)`, which needs $\text{Holding}(\mathit{Package})$ and $\text{At}(\mathit{Robot}, C)$.

$\text{Holding}(\mathit{Package})$ can only be produced by `PickUp(Package,l)` at a location l where the package already is — and the package only ever starts at A.

So the package must be picked up at A, carried $A \to B \to C$, and dropped.

The plan (4 actions)

```
PickUp(Package,A) ; Move(A,B) ; Move(B,C) ; Drop(Package,C)
```

State after every action

State	Action applied	Facts
$S_0 = I$	—	$\text{At}(Robot, A), \text{At}(Package, A)$
S_1	`PickUp(Package,A)`	$\text{At}(Robot, A), \text{Holding}(Package)$
S_2	`Move(A,B)`	$\text{At}(Robot, B), \text{Holding}(Package)$
S_3	`Move(B,C)`	$\text{At}(Robot, C), \text{Holding}(Package)$
S_4	`Drop(Package,C)`	$\text{At}(Robot, C), \text{At}(Package, C)$

$S_4 \supseteq G$, so $S_4 \models G$ and the plan is valid.

Note that $\text{At}(Package, A)$ disappears at step 1 (it is a negative effect of PickUp) and $\text{Holding}(Package)$ disappears at step 4 — the package is never in two places at once, which is what makes the delete lists necessary.

Why the sequence suggested in the handout does not work

The handout suggests reasoning about

`Move(A,B) ; PickUp(Package,B) ; Move(B,C) ; Drop(Package,C).`

This sequence is not a valid plan.

After `Move(A,B)` the state is

$$\{\text{At}(Robot, B), \text{At}(Package, A)\}.$$

`PickUp(Package,B)` requires $\text{At}(Package, B)$, which is false — the robot walked away and left the package behind.

The sequence has the right shape (move, pick up, move, drop) and is therefore easy to accept at a glance, which is precisely why every step must be checked against the state rather than against intuition.

This exact sequence is re-tested mechanically in Test C below, where the validator rejects it at step 2.

3. Task 2 — Asking an LLM to Implement the Planner

The LLM was given a specification, not the problem.

The prompt below was the exact text submitted (Claude was the model used); the domain itself was supplied separately, after the program had been produced, so that the LLM wrote a general planner rather than a hard-coded answer to the warehouse puzzle.

Exact prompt used

```
I want to implement a simple planning agent in Python.
Represent a state as a set of logical propositions.
Each action should contain:
- a name;
- positive preconditions;
- negative preconditions;
- positive effects;
- negative effects.
An action is applicable if all of its preconditions are satisfied by the current state.
When an action is applied:
1. remove its negative effects from the state;
2. add its positive effects to the state.
Use breadth-first search to find a sequence of actions that achieves a specified goal.
The program should also:
- detect when no plan exists;
- print the resulting sequence of actions;
- print the states reached after each action.
Explain the implementation and identify any assumptions you make.
```

What the LLM produced, and what had to be changed

The generated code supplied the Action class, applicable, apply_action and a breadth-first plan function, together with a written explanation.

Four changes were needed after reading and running it; all are marked (M1)–(M4) in `planner.py`:

- **(M1) validate_plan** — an independent verification routine that replays the plan from I and re-checks every precondition. The LLM was not asked for this and did not offer it; without it the only evidence that a plan is correct is the search's own bookkeeping, which is the thing under test.
- **(M2)** The negative precondition $\neg$Holding($Package$) on PickUp. The first version had an empty negative-precondition set everywhere, so the feature the specification asked for was never actually exercised.
- **(M3)** A frozenset-based visited set and deterministic sorting of successors. The first version had no closed list and looped forever on the cycle `Move(A,B) / Move(B,A)` — a real defect that only surfaced by running the code.
- **(M4)** The three test harnesses and the Inspect action used as an irrelevant action in Test C.

Think About It (Task 2) — where the specification appears in the code

Idea	Question it answers	Where it lives in `planner.py`
Preconditions	When is an action applicable?	`applicable(state, action)` — the single line `action.pos_pre <= state and not (action.neg_pre & state)`. This is the entailment test $S \models \mathrm{Pre}(a)$ and it is the only place the planner decides what is possible.
Effects	How does the state change?	`apply_action` — `new_state -= neg_eff` then `new_state |= pos_eff`, in that order. It raises `ValueError` on an inapplicable action, so no illegal transition can be produced even by a buggy search.
Goal	When does planning terminate?	`satisfies_goal(state, goal)` — the subset test `goal <= state`, called on the initial state and on every generated successor.
BFS	How are alternative plans explored?	`plan` — a FIFO deque frontier plus a visited set. The FIFO discipline is what makes the first plan found a shortest one; the visited set is what makes exhaustion of the frontier a proof that no plan exists.

4. Task 3 — Testing the Generated Planner

All output below is verbatim from `python3 planner.py`.

The program first prints its own applicability check for Task 0, which confirms the hand analysis in §1:

```
I = {At(Package,A), At(Robot,A)}
Drop(Package,A) applicable = False
Drop(Package,B) applicable = False
Drop(Package,C) applicable = False
Move(A,B) applicable = True
Move(B,A) applicable = False
Move(B,C) applicable = False
Move(C,B) applicable = False
PickUp(Package,A) applicable = True
PickUp(Package,B) applicable = False
PickUp(Package,C) applicable = False
```

Test A — Solvable problem (the original warehouse problem)

Initial state	$\{\text{At}(Robot, A), \text{At}(Package, A)\}$
Goal	$\{\text{At}(Package, C)\}$
Plan found?	YES — 4 actions
Resulting plan	`PickUp(Package,A) ; Move(A,B) ; Move(B,C) ; Drop(Package,C)`
Plan actually valid?	YES — independently re-executed; every precondition satisfied, final state entails the goal

```
Plan found : YES
Plan (4 actions):
1. PickUp(Package,A)
2. Move(A,B)
3. Move(B,C)
4. Drop(Package,C)

States reached after each action:
S0 = {At(Package,A), At(Robot,A)}
S1 = {At(Robot,A), Holding(Package)}
S2 = {At(Robot,B), Holding(Package)}
S3 = {At(Robot,C), Holding(Package)}
S4 = {At(Package,C), At(Robot,C)}

Independent validation:
[VALID] every precondition satisfied and the goal is entailed.
```

Verification of every action in the returned plan

Step 1: `PickUp(Package,A)` needs $\text{At}(Robot, A)$ ✓ and $\text{At}(Package, A)$ ✓ in S_0, and $\text{Holding}(Package) \notin S_0$ ✓.

Step 2: `Move(A,B)` needs $\text{At}(Robot, A)$ ✓ in S_1.

Step 3: `Move(B,C)` needs $\text{At}(Robot, B)$ ✓ in S_2.

Step 4: `Drop(Package,C)` needs $\text{At}(Robot, C)$ ✓ and $\text{Holding}(Package)$ ✓ in S_3.

Finally,

$$\text{At}(Package, C) \in S_4,$$

so $S_4 \models G$.

The plan is identical to the one constructed by hand in §2, including its length — which is expected, since BFS with unit action costs returns a shortest plan.

Test B — Impossible problem (PickUp removed)

Initial state	$\{\text{At}(Robot, A), \text{At}(Package, A)\}$
Goal	$\{\text{At}(Package, C)\}$
Modification	All three PickUp actions removed from A
Plan found?	NO — reported "No plan found"
Resulting plan	— (none returned)
Behaviour correct?	YES — no action was invented

```
Actions     : Drop(Package,A), Drop(Package,B), Drop(Package,C),
              Move(A,B), Move(B,A), Move(B,C), Move(C,B)
Plan found : NO
No plan found
```

This is the correct result, and it is a proof rather than a timeout: without PickUp, Holding($Package$) can never become true, so Drop is never applicable, so $\text{At}(Package, A)$ can never be deleted and $\text{At}(Package, C)$ can never be added.

The reachable state space is finite (the robot's three possible locations), the visited set prevents revisiting, so the frontier empties after a handful of expansions and the planner returns `None`.

Critically the planner does not fabricate a "Teleport" or "Carry" action to close the gap — the action set is the only source of transitions, and the search cannot step outside it.

Test C — Irrelevant actions

Three `Inspect(l)` actions were added. Each requires $\text{At}(Robot, l)$ and adds `Inspected(l)`; they change the state and therefore genuinely enlarge the search space, but they can never contribute to $\text{At}(Package, C)$.

The Move actions are also available while the package stays at A, which is the confusion the test is designed to expose.

C1 — control: goal $\text{At}(Robot, C)$

Initial state	$\{\text{At}(Robot, A), \text{At}(Package, A)\}$
Goal	$\{\text{At}(Robot, C)\}$
Plan found?	YES — 2 actions: `Move(A,B) ; Move(B,C)`
Plan actually valid?	YES

Final state:

$$\{\text{At}(Package, A), \text{At}(Robot, C)\}.$$

The robot is at C and the package is still at A. This is the state that a planner conflating the two goals would wrongly accept.

C2 — the real goal $\mathrm{At}(Package, C)$, with the irrelevant actions still available

Initial state	$\{\mathrm{At}(Robot, A), \mathrm{At}(Package, A)\}$
Goal	$\{\mathrm{At}(Package, C)\}$
Plan found?	YES — 4 actions
Resulting plan	`PickUp(Package,A) ; Move(A,B) ; Move(B,C) ; Drop(Package,C)`
Plan actually valid?	YES

```
Final state contains At(Package,C) : True
Plan contains a PickUp action     : True
Plan contains a Drop action       : True
Plan contains no Inspect action   : True

Validating the bogus plan [Move(A,B), Move(B,C)] against goal At(Package,C):
[INVALID] final state does not entail the goal; missing ['At(Package,C)']

Validating the 'looks reasonable' plan
[Move(A,B), PickUp(Package,B), Move(B,C), Drop(Package,C)]:
[INVALID] step 2: PickUp(Package,B)
         unsatisfied positive preconditions: ['At(Package,B)']
```

Three things are established here.

First, the 2-action robot-only plan of C1 is not returned for the package goal — the planner does not treat the robot reaching C as the package reaching C, because the goal test is a subset test on distinct atoms and

$$\mathrm{At}(Robot, C) \neq \mathrm{At}(Package, C).$$

Second, the irrelevant Inspect actions are explored but never appear in the returned plan, since BFS returns a shortest plan and no Inspect action is ever needed.

Third, the validator rejects the handout's plausible-looking sequence at exactly the right step and names the missing atom.

5. Task 4 — Logic and Search

The planner rests on two distinct mechanisms, and they do different jobs.

Logical reasoning answers a question about a single state–action pair: given a state S and an action a, does

$$S \models \text{Preconditions}(a)?$$

If it does, the transition

$$S' = \text{Apply}(S, a)$$

is licensed, and the add/delete lists determine exactly which atoms change.

Logic is also what ends the process: the goal test $S \models G$ is an entailment check.

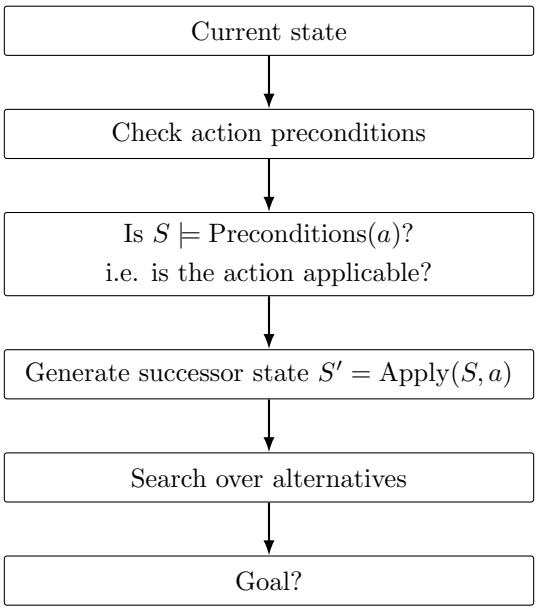
In the code, logic is confined to three small functions — `applicable`, `apply_action` and `satisfies_goal` — none of which knows that a search is happening.

Search answers a question about the space of states: of the many actions that are logically applicable in the states reached so far, which sequence should be tried next?

Logic can tell us that `PickUp(Package,A)` and `Move(A,B)` are both applicable in I; it cannot tell us which one leads to the goal.

That is the search's job, and here it is breadth-first: a FIFO frontier, a visited set, and expansion of every applicable action at each state.

Completed description



The missing step is the applicability decision: the entailment test whose answer determines whether a successor is generated at all.

Inapplicable actions are discarded here and never enter the frontier, so the search only ever explores states that are genuinely reachable.

How the two work together

The loop is:

1. take a state from the frontier;
2. for each action in A, ask logic whether it is applicable in that state;
3. for those that are, ask logic what the successor state is;
4. hand those successors to the search, which queues them;
5. discard ones already seen;
6. test each against the goal;
7. repeat until a goal state is reached or the frontier empties.

Neither half suffices alone.

Logic without search knows every legal move but has no idea which to make, and would either stall or wander.

Search without logic explores sequences with no guarantee that any of them can actually be executed — it would happily return

```
Move(A,B) ; PickUp(Package,B) ; ...
```

because the sequence looks like progress.

Soundness comes from logic (every action in the returned plan is executable at the point it is executed); completeness and optimality come from search (BFS exhausts shorter plans before longer ones, so if a plan exists a shortest one is found, and if the frontier empties no plan exists).

Think About It (Task 4)

> *"Logic determines what is possible; search determines what to try."*
> This separation is why the planner is the same algorithm as the uninformed searches from the previous module, with one substitution.
> A graph search needs a successor function; here the successor function is not a table of edges but is computed from a logical description — the successors of S are exactly
>
> $$\{\text{Apply}(S, a) : a \in A,\ S \models \text{Pre}(a)\}.$$
>
> The state space is never built explicitly; it is generated lazily by entailment.
> Everything else — FIFO frontier, visited set, path reconstruction, shortest-path property — is unchanged from ordinary BFS.
> Swapping the frontier for a priority queue with a heuristic would give A* over the same logically-defined space.

6. Task 5 (optional) — Can the LLM Verify Its Own Plan?

The LLM was asked:

> "For every action in the plan, identify its preconditions and show that those preconditions are satisfied in the state in which the action is executed."

It produced a fluent, correctly formatted, step-by-step justification of the four-action plan.

Compared against the state transitions computed independently by `trace_states` and re-checked by `validate_plan`, the explanation happened to agree.

Which should be trusted more?

(b), the independently executed state transitions.

The explanation and the plan come from the same process, so an error in the plan is likely to be reproduced rather than caught by the explanation — the LLM is being asked to audit itself using the same reasoning that produced the artefact.

More fundamentally, the explanation is generated text: it is optimised for plausibility, and its persuasiveness is uncorrelated with its correctness.

The Python execution is an independent check with a different failure mode — it re-derives each state mechanically from I and the add/delete lists, and it will reject a step whose preconditions are unmet however convincingly that step is described.

Notice that agreement in this instance is evidence, not proof: the same explanation machinery also produced a confident-sounding four-action sequence in §2 that the validator rejects at step 2.

The general principle is the one the handout states — a generated explanation is not the same as an independent verification — and the corresponding architecture is

$$\boxed{\text{generate} \rightarrow \text{independently verify}}$$

where the verifier must not share the generator's assumptions.

7. Reflection Questions

1. Why is it useful to specify action preconditions and effects before asking an LLM to write the planner?

Because it converts an open-ended request into a specification with a checkable answer.

Asking an LLM to "solve the warehouse problem" invites it to produce the four-action sequence directly from pattern recognition — plausible, unverifiable, and useless for any other problem.

Specifying states as sets of atoms, and actions as (pos-pre, neg-pre, add, delete), fixes the data structures and therefore fixes what "correct" means: I could then check the code against the specification rather than against my intuition about warehouses.

It also forced my own understanding to be complete first — writing the precondition table in §1 is what revealed that the gripper constraint was unstated.

And it yields a general planner: the same code accepts the modified domains of Tests B and C without edits.

2. Give an example of an error that could occur if the planner failed to check an action's preconditions.

Taking the handout's suggested sequence

```
Move(A,B) ; PickUp(Package,B) ; Move(B,C) ; Drop(Package,C)
```

without precondition checking, `PickUp(Package,B)` would be applied in the state

$$\{\text{At}(Robot, B), \text{At}(Package, A)\}.$$

Its delete list removes $\text{At}(Package, B)$ — which is not present, so the deletion silently does nothing — and its add list asserts $\text{Holding}(Package)$.

The package is now simultaneously at A and in the robot's gripper.

`Move(B,C)` and `Drop(Package,C)` then apply cleanly and the planner reports success with $\text{At}(Package, C)$ and $\text{At}(Package, A)$ both true.

The output is a plan that a real robot would execute by closing its gripper on empty air at B and "dropping" nothing at C, while the package sits untouched at A.

The planner reports no error, because without the entailment test nothing in the program is ever in a position to notice.

3. Why is a plan that "looks reasonable" not necessarily a valid plan?

Because "reasonable" is a judgement about the shape of the sequence — go there, pick it up, carry it, put it down — while validity is a claim about a chain of state transitions, each of which must be licensed by the state that actually obtains at that point.

The two come apart exactly when an intermediate state differs from the one we imagined.

The handout's sequence has a perfect narrative shape and fails at step 2 on a single missing atom, $\text{At}(Package, B)$.

Surface plausibility is also the thing LLMs optimise, which makes it a particularly unreliable signal here.

Validity is only established by re-executing the plan and re-checking each precondition against the computed state — which is what `validate_plan` does, and why it was written by hand rather than trusted to the generator.

4. What did the LLM contribute to the implementation?

The engineering scaffolding: the Action class with its four fields; the applicable entailment test as set operations

```
pos_pre <= state,    neg_pre & state;
```

the delete-then-add `apply_action`; the subset goal test; and the overall BFS skeleton with a deque

frontier and path accumulation.

It also produced a readable explanation and listed some of its own assumptions.

What it contributed was fast, idiomatic translation of a specification I had already written into working Python — it did not contribute the specification, the domain model, or any assurance that the result was right.

5. What did you have to verify independently?

Everything that mattered.

Concretely:

(i) the cycle handling — the first generated version had no visited set and did not terminate on `Move(A,B)/Move(B,A)`, found only by running it;

(ii) that the negative-precondition machinery was actually exercised, which required adding $\neg$ Holding($Package$) to PickUp;

(iii) that "no plan" is reported rather than an action invented (Test B);

(iv) that At($Robot, C$) is not conflated with At($Package, C$) (Test C);

(v) that every action of the returned plan is applicable in the state where it is executed, and that the final state entails the goal — re-derived by `validate_plan` independently of the search's own bookkeeping;

(vi) that the returned plan matches the plan I had constructed by hand in §2 before running anything.

The hand-built plan mattered here: it gave an answer to compare against that the LLM had no part in producing.

6. Where is logical reasoning being used in this laboratory?

In four places.

(a) Applicability:

$$S \models \text{Preconditions}(a),$$

the entailment test in `applicable` — the closed-world assumption is what makes the negative preconditions decidable by a set intersection.

(b) The transition function:

the add/delete lists are a syntactic specification of how truth values change, and delete-then-add ordering is part of the semantics.

(c) Termination:

the goal test

$$S \models G$$

as a subset check.

(d) The impossibility argument in Test B,

which is a genuine logical proof that no plan exists — not merely a failure to find one — because Holding(*Package*) is underivable once PickUp is removed.

Logic is also what I used by hand in §2 to reason backwards from the goal to the required first action.

7. How is planning related to the search algorithms studied in the previous module?

Planning is graph search, over a graph that is described logically instead of being given explicitly.

States are nodes; applicable actions are edges; a plan is a path from I to any node satisfying G.

The one structural difference is that the successor function is computed rather than stored:

$$\text{successors}(S) = \{\text{Apply}(S, a) : a \in A,\ S \models \text{Pre}(a)\}.$$

The successors are generated lazily, so the state space is never materialised — which matters because it grows exponentially in the number of atoms.

Everything else transfers directly:

- the FIFO frontier gives BFS its shortest-plan guarantee under unit action costs;
- the visited set handles the cycles created by `Move(A,B)/Move(B,A)`;
- the visited set gives the termination that makes "no plan found" meaningful;
- replacing the queue with a priority queue and a heuristic (for example the number of unsatisfied goal atoms) gives A* over the same space.

The planning-specific contribution is not a new search algorithm but the logical representation that generates the graph — and, with it, domain-independent heuristics that can be read off the action descriptions.

8. Statement of LLM Use

Component	Origin	Notes
Problem specification (§1), hand plan (§2)	Written by hand	Produced before the LLM was used, as Task 0 and Task 1 require.
Action class	LLM-generated	Used essentially unchanged.
`applicable`, `apply_action`, `satisfies_goal`	LLM-generated	The `ValueError` guard in `apply_action` was added by hand.
`plan` (BFS)	LLM-generated, hand-modified	(M3) visited set and deterministic action ordering added by hand; the generated version did not terminate on cyclic Move actions.
Negative precondition on PickUp	Hand-added (M2)	Assumption A2; not stated in the laboratory text.
`validate_plan`, `trace_states`	Written by hand (M1)	Deliberately independent of the search, so that verification does not reuse the generator's bookkeeping.
`run_tests`, Inspect, report	Written by hand (M4)	Tests A, B, C and their cross-checks.
This report	Written by hand	All analysis, test design and reflection answers are my own; the quoted program output is verbatim from running `planner.py`.

Companion file: `planner.py` — run with

```
python3 planner.py
```

to reproduce every figure quoted in §4.